

Decentralized Computing and Blockchains

Other Blockchains

Alysson Bessani
DI-FCUL

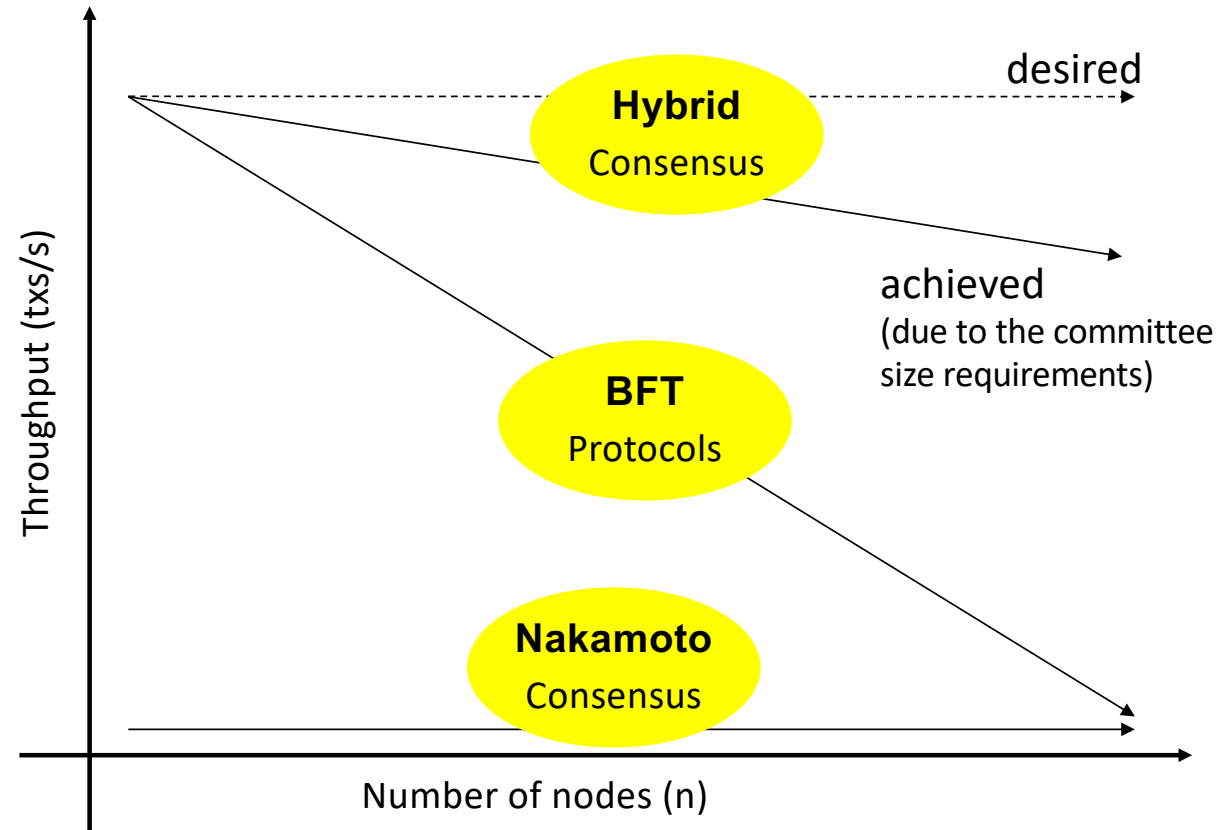
Outline

- Hybrid Consensus (a fundamental idea)
- Solana (a high-profile Ethereum competitor)
- Hyperledger Fabric (one of the most popular consortium ledger)

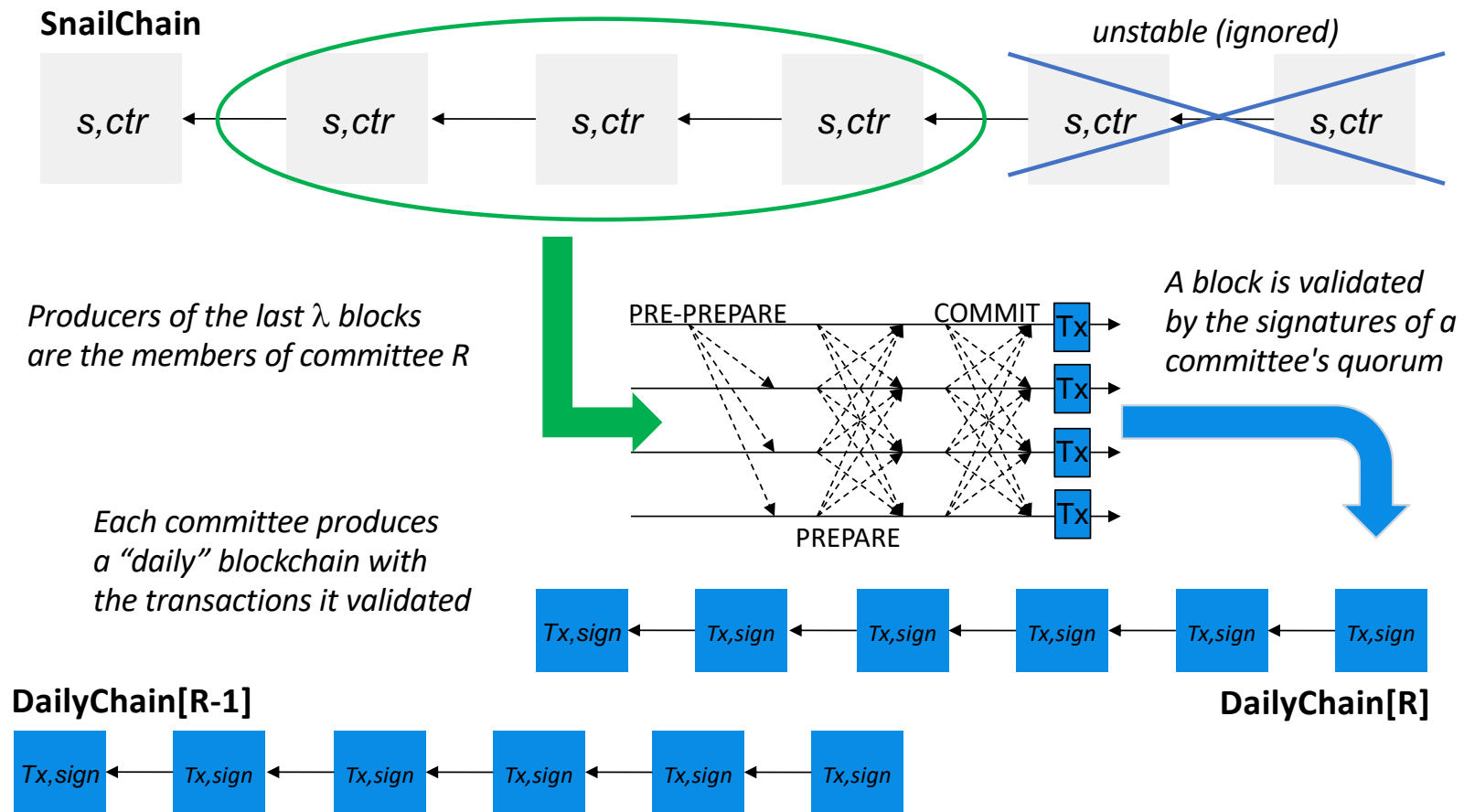
Hybrid Consensus

- The idea of a committee is typically used in permissionless PoS blockchains
- However, it can be used also to accelerate PoW blockchains, i.e., to build permissionless blockchains based on the use of both Nakamoto and (traditional) BFT consensus together
- We are particularly interested on the idea of using two blockchains:
 - One for defining a committee of members to participate in the ordering of transactions
 - Other, maintained by the committee, used for ordering transactions
 - Observation:
 - This idea was introduced in Bitcoin-NG, in which two blockchains (with different PoW difficulties) were used
 - It is possible to define committee members without using a blockchain by requiring a PoW for participating for a time (e.g., as in Solida)

Why Hybrid Consensus?



Hybrid Consensus: General Idea



Hybrid Blockchains

- This protocol achieves responsiveness, i.e., the commit latency of a transaction is proportional to the network latency
 - Nakamoto consensus is not responsive (latency depends on the time to solve the PoW) and classical BFT protocols (e.g., HotStuff) are typically responsive (latency depends on network latency)
- Required assumptions (for responsiveness):
 - Synchronous system (as in Nakamoto consensus)
 - This work also shows that (1) PoW-based permissionless consensus is impossible in partially synchronous systems, and (2) the peers need to continuously execute PoW for the snailchain
 - Less than 1/3 of Byzantine nodes
 - Corrupting a node takes at least some time τ
- Practical observation: the size of the committee (λ) and the duration of the “day” are dependent of τ



Solana

Another high-profile PoS blockchain

Solana

- Solana is a blockchain with similar aims to Ethereum, i.e., it wants to be a World Computer
- Launched in 2020, targeting blockchain scalability, with a PoS consensus that has a theoretical throughput of up to 65K txs/s
- It's low inter-block time (0.4s) makes Solana one of the preferred platforms for high-frequency applications like decentralized exchanges and gaming
- Solana in 2026:
 - 50% of global DEX (Decentralized Exchanges) are processed in Solana
 - New consensus protocol (Alpenglow) enable low latency transaction processing (150ms), enabling high-frequency trading (trading bots)
 - Currently, it has more tokenized real-world assets than Ethereum
 - Low fees and high performance is key for supporting real-world applications like NFTs and blockchain games

Solana

- The Sol cryptocurrency is used in the same way as Ether in Ethereum
- Smart contracts (programs) are written mostly in Rust (also in C and C++)
 - Programs are compiled to bytecode and executed on Solana Virtual Machines (SVM)
 - Allows the parallel execution of smart contracts (by requiring transactions to declare which accounts they will be using)
- Solana is a PoS blockchain, just like Ethereum 2 <https://coinmarketcap.com/currencies/solana/>
- To be a validator, a node needs
 - High-performance hardware
 - $\geq 512\text{GB}$ of RAM, fast storage
 - Validated using runtime performance
 - To commit some variable stake
 - Spent as it votes



Solana Operation (newer Alpenglow version)

- The system is open (permissionless), but uses a normal (permissioned) consensus called Alpenglow on each **epoch**
- An epoch is expected to have 18000 **slots**, each with 0.4 secs
 - Each slot has a leader that can generate a block
 - In case of failures and asynchrony, less blocks will be generated
 - An epoch should take about 2 hours
- **Nodes register the interest of participating on an epoch e** , by issuing a transaction with its information (public key, IP:port, and stake) during epoch $e-2$
 - At the end of epoch $e-2$, the validators of epoch e are defined together with their stake weight (voting power)
 - The maximum number of validators is $n_{\max} = 2000$
 - Before the epoch starts, the system defines a **schedule of leaders** for the epoch using a verifiable random function. Each leader is responsible for 4 slots (a window)

Alpenglow

- Alpenglow is remarkably similar to DispersedSimplex
 - It uses erasure code to balance bandwidth consumption
 - It can finalize a block in a single slot/view
- The main difference is that it decouples the block proposal from the finalization protocol and incorporates a fast path in the latter
 - Making it similar to a recent protocol called Kudzu (see refs)
- Two main components:
 - Rotor: Block dissemination using erasure codes
 - Votor: Fast finalization protocol

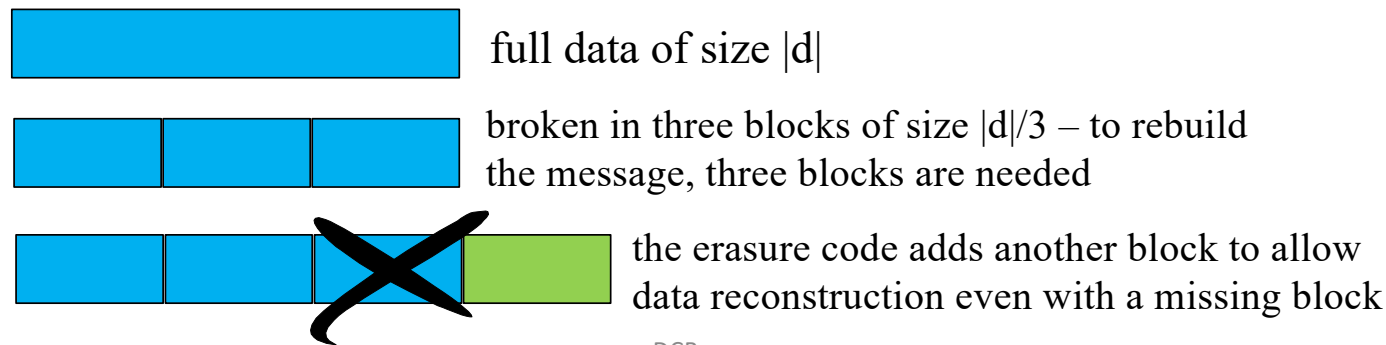
Alpenglow: model and assumptions

- During an epoch, there will be n nodes, each with its stake
- They claim a static adversary (during the epoch) can control:
 - $< 20\%$ of the total stake, which can be subject to Byzantine failures, plus
 - $< 20\%$ of the total stake, which can be unavailable (subject to crashes)
 - In fact, Alpenglow supports a fast path with up to 20% Byzantine faults, or alternatively tolerates up to 40% crash faults but no Byzantine faults
- Partially-synchronous system, i.e., asynchronous system until (unknown) GST, then synchronous with a maximum known delay Δ
 - No synchronized clocks
 - Protocol is always safe, but live only in synchronous periods

(From a previous lecture)

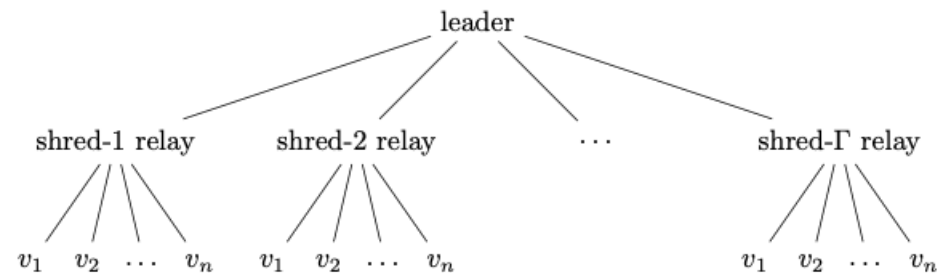
Erasure Codes

- (k,n) -erasure coding is a form of forward error correction, one of the many types of error correction codes
- A message is broken in k blocks
- Extra $m=n-k$ blocks are added so that even if m blocks are lost, the message can still be reconstructed
- This incurs additional data overhead, but avoids ACKs, timeouts, retries (for messages) and file unavailability or corruption
- Example: $n = 4$ and $k = 3$



Rotor: Erasure Code in Alpenglw

- Blocks are divided in slices
- For each slice, the leader generates n fragments (shreds) and Merkle-tree-based slice certificate
- Each shred fits in an UDP packet
- Each shred is sent to a randomly-selected relay, which retransmit it to all nodes
- To work well, Rotor requires erasure coding with $n/k = 5/3$



- The 20+20 failure model of Alpenglw works only if the following holds:
- *If a correct node receives a full block B via Rotor, any other correct node that receives a full block via Rotor for the same slot, receives the same block B*

Votor: Fast finalization of Blocks

- Once a node obtains the slot block (via Rotor) to a node, it verifies it and, assuming everything is ok, it broadcasts a notarization vote for it
- Once a node observes notarization votes from 60% of the stake, it builds a notarization certificate and broadcasts a finalization vote
- Once a node observes notarization votes from 80% of the stake, it builds a fast finalization certificate and **the block is finalized**
- Once a node observes finalization votes from 60% of the stake, it builds a finalization certificate and **the block is finalized**
- If there is a timeout, the node vote for skipping a window; once it observes skipping votes from 60% of the stake, it moves on

Some numbers about the Solana network

<https://solscan.io/analytics>

Transactions

508,434,696,635

All time

Transaction Per Second (TPS)

3,204.37 ↑ 4%

In the last 24H

Total Network Fees (SOL)

5,270.958 ↑ 25.07%

Non-vote Transactions fee in the last 24h

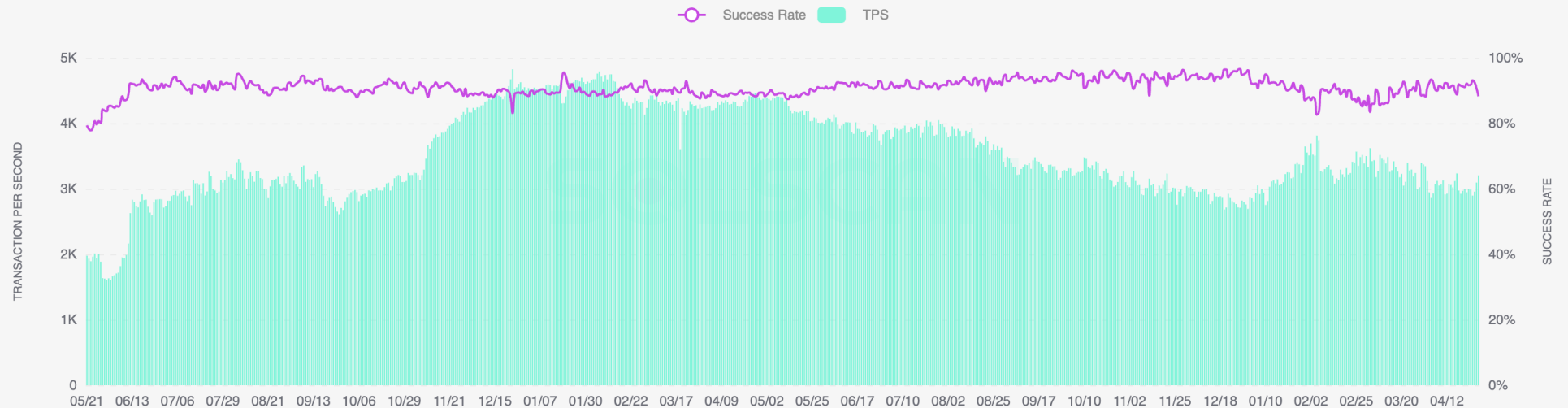
Validators

774

Voted in the last 24h

Transactions per Second (TPS) and Success rate

7 Days 1 Month 3 Months **All time** 





HYPERLEDGER **FABRIC**

<https://www.lfdecentralizedtrust.org/projects/fabric>

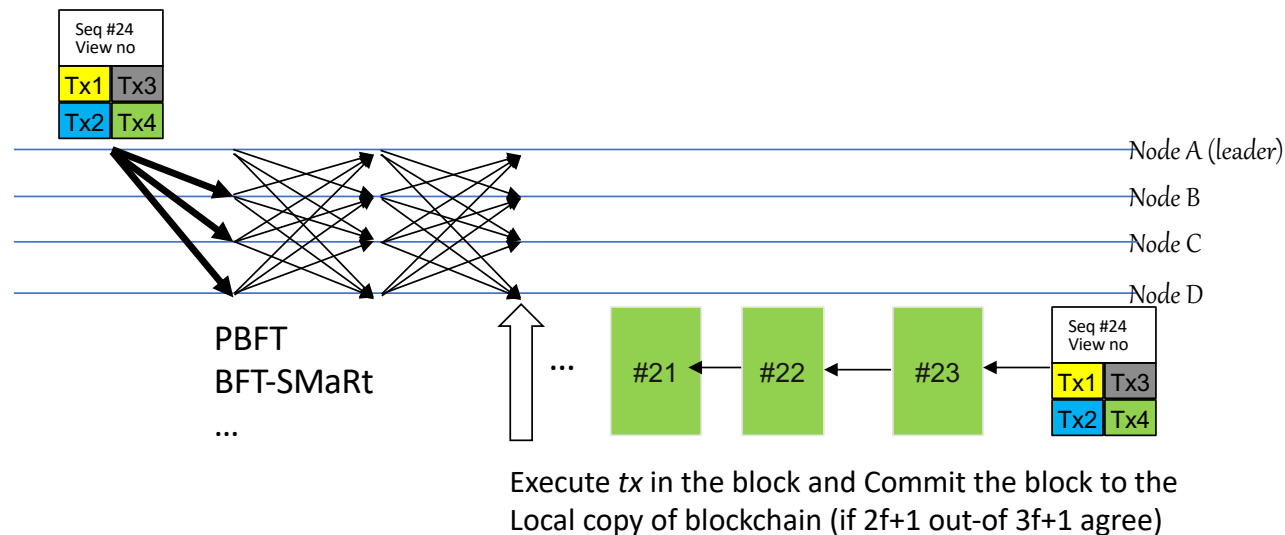
(From a previous lecture)

Permissioned Ledgers

- To participate, a party needs to have a verifiable identity (ID) and be accepted in the network
 - This ID usually is accompanied with a certified public-private key pair, used to establish authenticated channels
 - Sybil attacks are not a concern with verifiable IDs
- Not required to scale to thousands of nodes
 - E.g., 100 may be enough...
- Employ traditional Byzantine consensus protocols:
 - All nodes know all other nodes (as well as n and f)
 - The adversarial threshold is structural: the protocol (sometimes called *consortium consensus*) works if the number of faulty nodes is less than a third of the total nodes ($n > 3f$)
- This is the blockchain architecture preferred by existing companies as it removes intermediaries and is not related to dark web or illegal activities
 - Sometimes it is called DLT (Distributed Ledger Technology) to differentiate from open blockchains such as Ethereum

(Traditional) Permissioned Blockchain Architecture

(e.g., Quorum, Tendermint)



1. Inputs to the state machine (smart contract txs) are totally ordered
2. Executed in sequence, after consensus (ordering)
3. Most permissioned blockchains are architected like this

Issues with (Traditional) Permissioned Blockchain Architecture

- **Sequential execution of smart contracts invocations**
 - Execution latency blocks other smart contracts, hampers performance
 - DoS against smart contracts (e.g., 'while true { }')
 - How permissionless blockchains cope with it?
 - Gas (paying for every step of computation)
 - Tied to a cryptocurrency 
- **Non-determinism**
 - Smart-contracts must be deterministic (otherwise – state forks)
 - How permissionless blockchains cope with it:
 - Enforcing determinism: Solidity DSL, Ethereum VM 
 - Cannot code smart-contracts in general-purpose languages
- **Confidentiality of execution: all nodes execute all smart contracts**
- **Inflexible consensus: consensus protocols are hard-coded**
- **Inflexible trust-model: exposing low-level consensus assumptions**

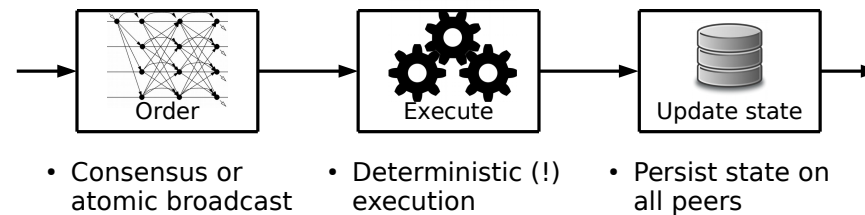
Fabric Key Requirements

- **No native cryptocurrency**
- **Smart-contracts in general-purpose languages**
- **Modular/pluggable consensus**

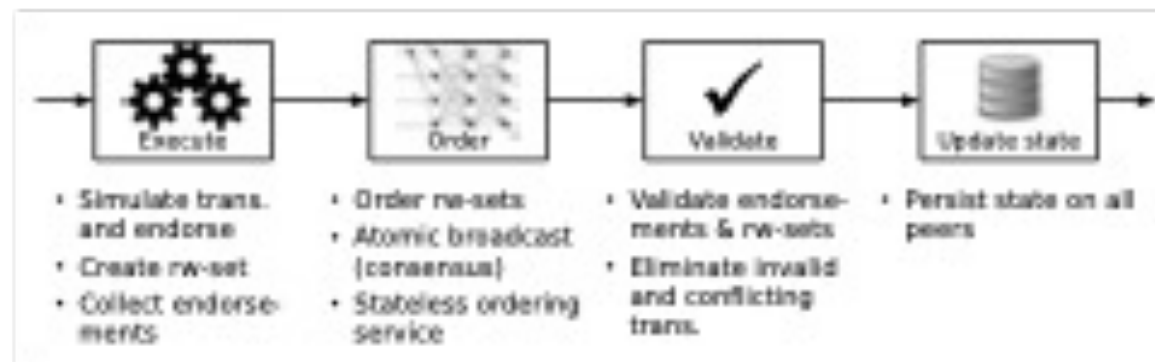
Satisfying these requirements required a complete overhaul of the permissioned blockchain design!

Traditional vs. Fabric Models

- Traditional



- Fabric

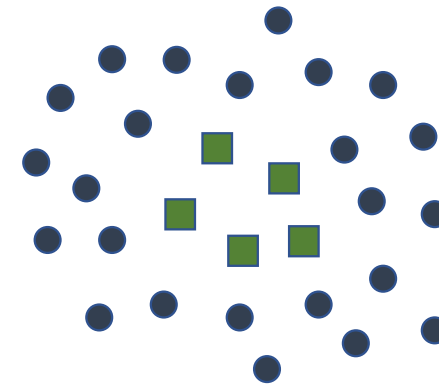




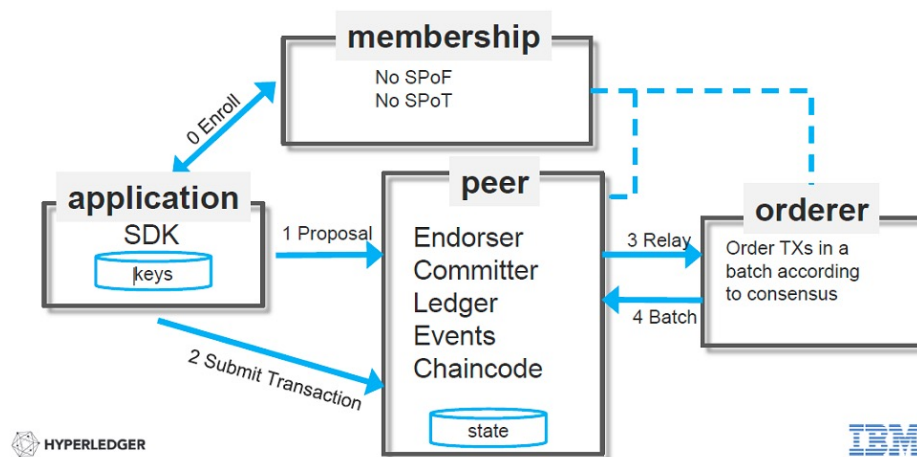
- Modular platform for permissioned blockchains
- Enables clients to manage transactions by using chaincodes, peers and an ordering service
 - Chaincode: Fabric's counterpart for smart contracts
 - Can be written in any language
 - Work on a versioned key-value store
 - Peers: maintain the ledger/database and execute chaincodes and validate results
 - Ordering service: creates blocks of transactions and define their order in the distributed ledger

HYPERLEDGER FABRIC

■ orderer
● peer

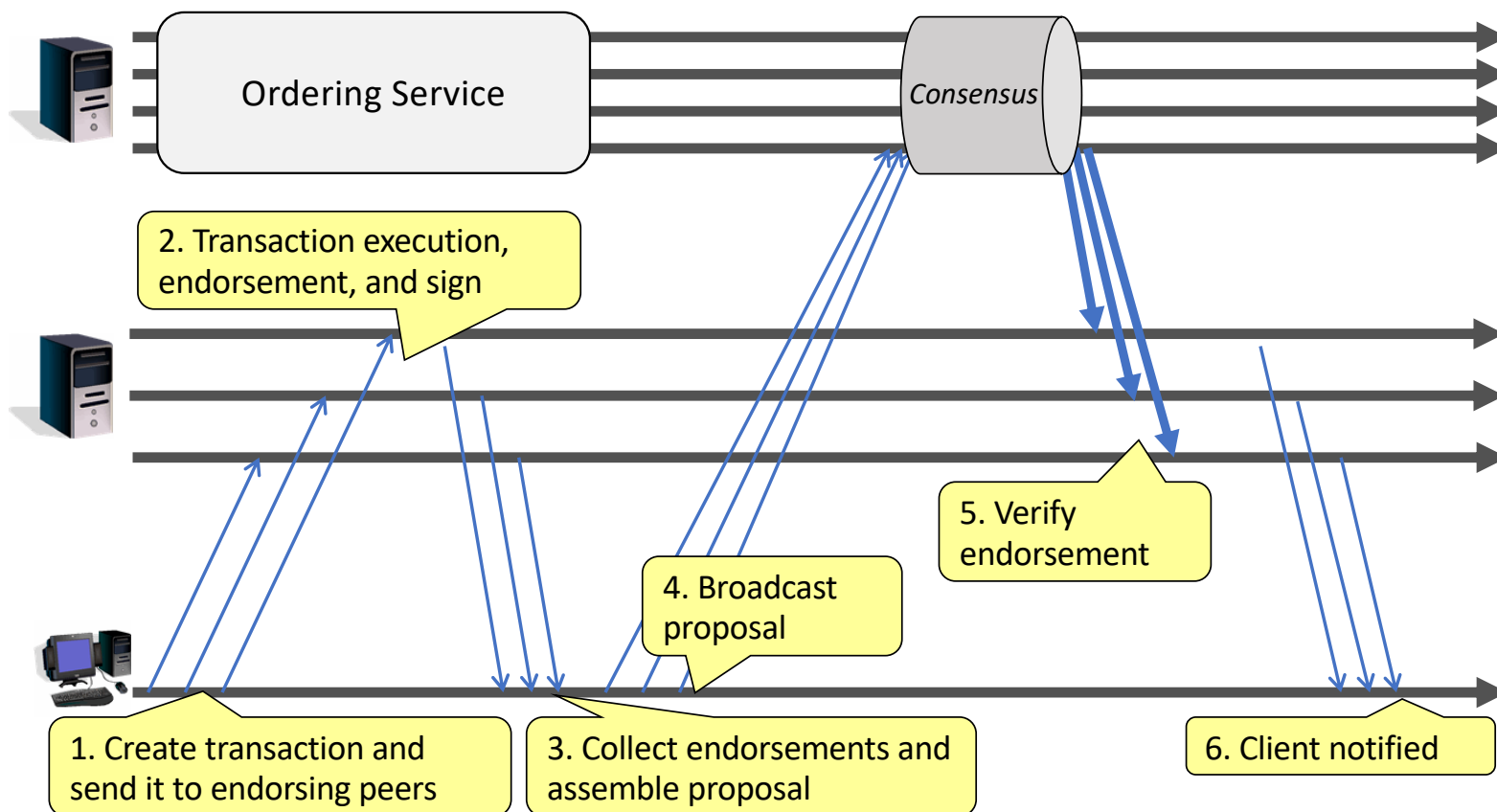


Fabric v1.0 Architecture





HYPERLEDGER FABRIC

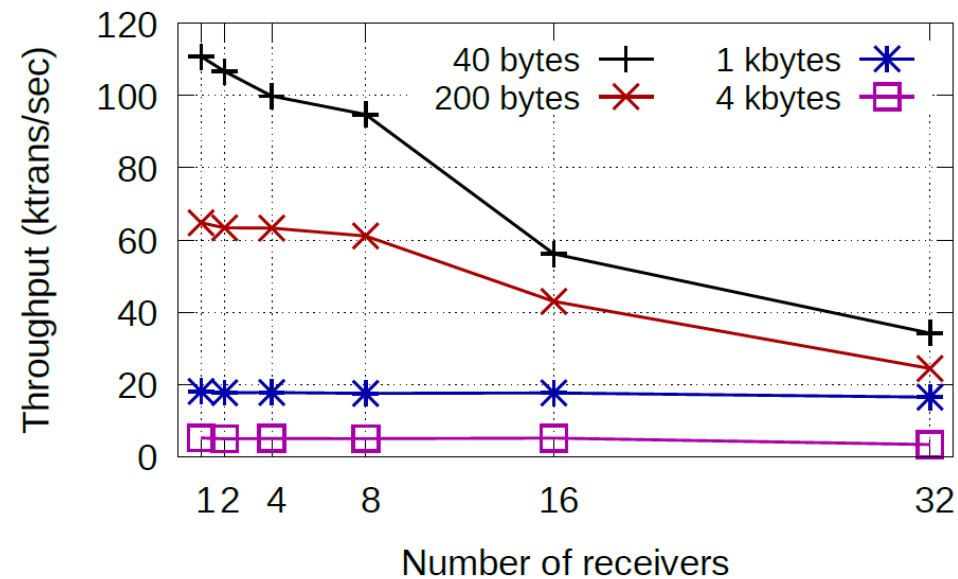




- Fabric supports different ordering services
- Version 3 provides two:
 - RAFT-based module, a crash fault-tolerant orderer based on etcd's implementation of the RAFT protocol;
 - A BFT ordering service based on BFT-SMaRt, but translated to the Go programming language and improved by IBM Israel.
- I'll present some results for the BFT ordering service we prototyped using BFT-SMaRt in 2018.



- Performance w/ BFT Orderer (based on BFT-SMaRt)
 - 4 nodes
 - Variable number of peers (receivers)
 - Different transaction sizes



(some) Bibliography

- Q. Kniep et al. **Solana Alpenglouw Consensus**. White Paper v1.1. 2025.
- N. Shrestha et al. **Hydrangea: Optimistic Two-Round Partial Synchrony with Improved Fault Resilience**. <https://eprint.iacr.org/2025/1112.pdf>
- V. Shoup et al. **Kudzu: Fast and Simple High-Throughput BFT**. DISC'25.
- E. Androulaki et al. **Hyperledger Fabric: A Distributed Operating System for Permissioned Blockchains**. EuroSys'18.
- J. Sousa et al. **A Byzantine Fault-Tolerant Ordering Service for the Hyperledger Fabric Blockchain Platform**. DSN'18.
- R. Pass and E. Shi. **Hybrid Consensus: Efficient Consensus in the Permissionless Model**. DISC'17.